

Divide & Conquer

Smitasankhe@somaiya.edu

Algorithmic Evaluation

Strategies are evaluated along the following dimensions:

- Completeness : does it always find a solution if one exists?
- Time complexity : number of nodes generated
- Space complexity : maximum number of nodes in memory
- Optimality : does it always find a least-cost solution?

Divide-and-conquer

- Breaking the problem into several sub-problems that are similar to the original problem but smaller in size.
- Solve the sub-problem recursively (successively and independently), and then
- Combine these solutions to sub-problems to create a solution to the original problem.

Control Abstraction

Type DAndC(Problem P)

```
{  
if small (P) return S(P);  
else {  
  divide P into smaller instances P1, P2, .... ,Pk,  $k \geq 1$ ;  
  Apply DAndC to each of these sub problems;  
  Return combine(DAndC(P1), DAndC(P2),.....,  
    DAndC(Pk));  
}
```

General Form

$$T(n) = aT(n/b) + f(n)$$

Where,

n: size of original problem.

a: number of subproblems.

b: size of each subproblem.

f(n): time to divide and combine subproblems

$$T(n) = \begin{cases} T(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

Finding Maximum and Minimum

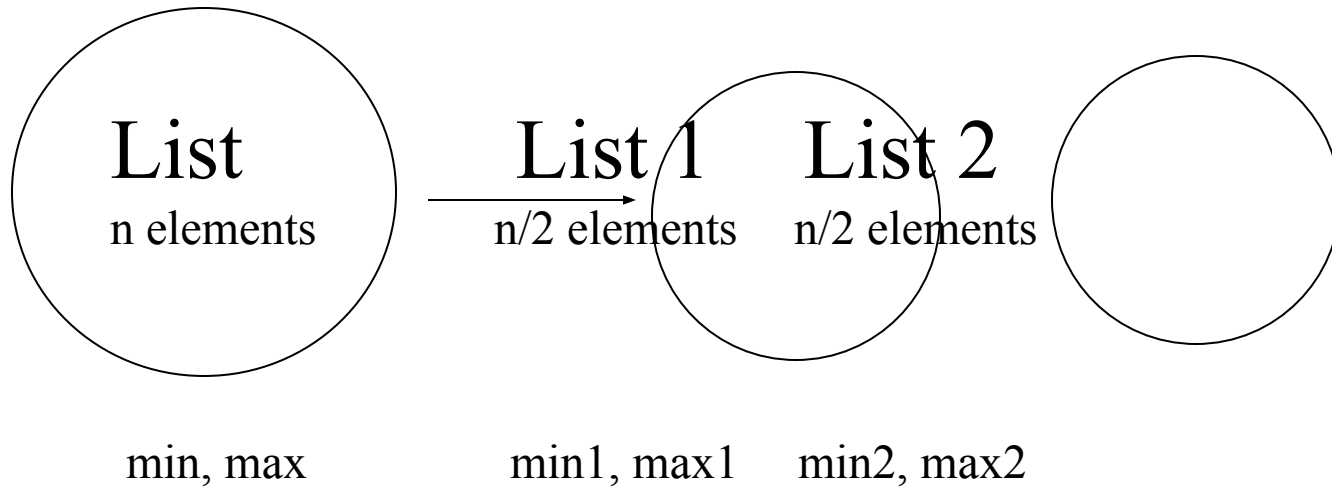
```
1  Algorithm StraightMaxMin( $a, n, max, min$ )
2  // Set  $max$  to the maximum and  $min$  to the minimum of  $a[1 : n]$ .
3  {
4       $max := min := a[1]$ ;
5      for  $i := 2$  to  $n$  do
6          {
7              if ( $a[i] > max$ ) then  $max := a[i]$ ;
8              if ( $a[i] < min$ ) then  $min := a[i]$ ;
9          }
10 }
```

1. Find the maximum and minimum

The problem: Given a list of unordered n elements, find max and min

The straightforward algorithm:

```
max  $\leftarrow$  min  $\leftarrow$  A (1);  
for  $i \leftarrow 2$  to  $n$  do  
    if A ( $i$ )  $\triangleright$  max, max  $\leftarrow$  A ( $i$ );  
    if A ( $i$ )  $\triangleleft$  min, min  $\leftarrow$  A ( $i$ );
```

min = MIN (min1, min2)
max = MAX (max1, max2)

```

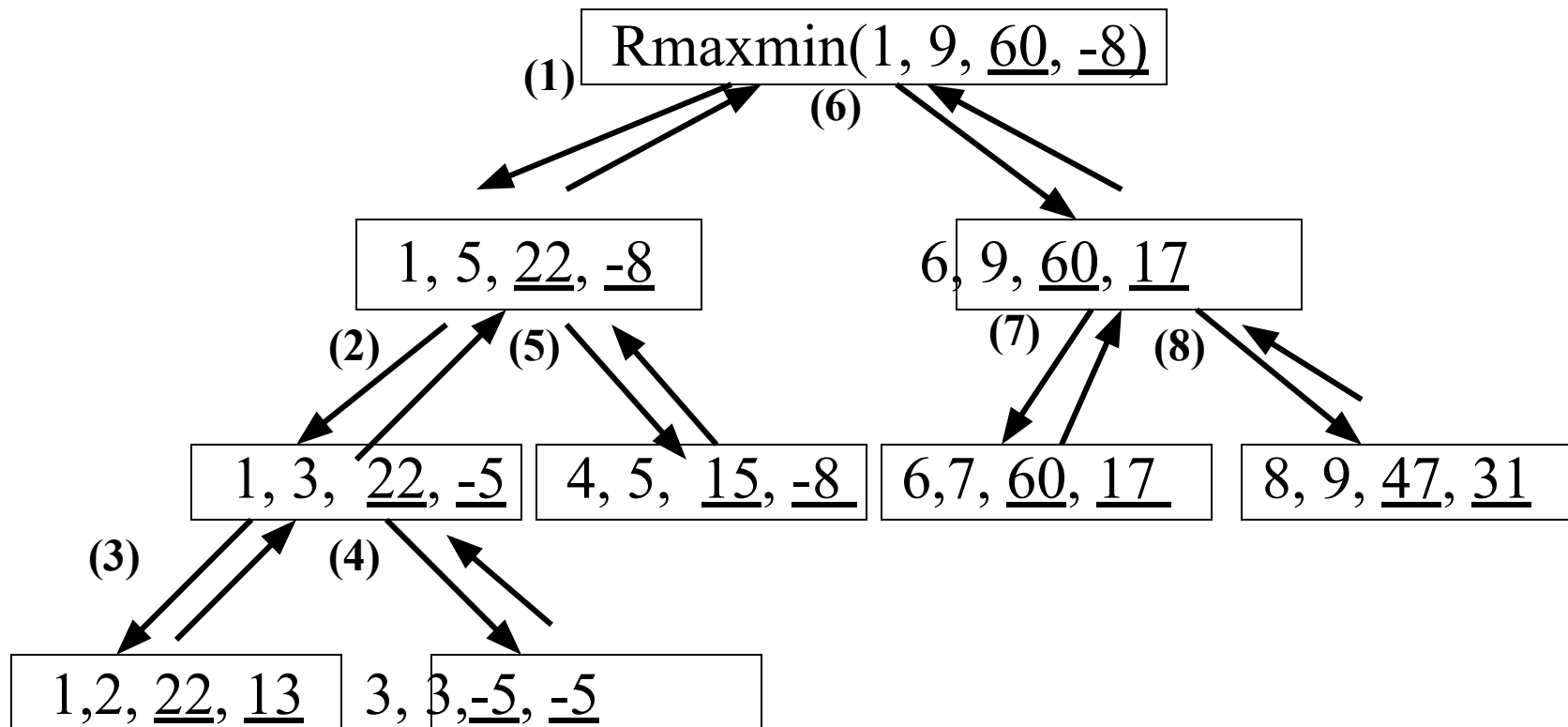
1  Algorithm MaxMin( $i, j, max, min$ )
2  //  $a[1 : n]$  is a global array. Parameters  $i$  and  $j$  are integers,
3  //  $1 \leq i \leq j \leq n$ . The effect is to set  $max$  and  $min$  to the
4  // largest and smallest values in  $a[i : j]$ , respectively.
5  {
6      if ( $i = j$ ) then  $max := min := a[i]$ ; // Small( $P$ )
7      else if ( $i = j - 1$ ) then // Another case of Small( $P$ )
8          {
9              if ( $a[i] < a[j]$ ) then
10                 {
11                      $max := a[j]$ ;  $min := a[i]$ ;
12                 }
13             else
14                 {
15                      $max := a[i]$ ;  $min := a[j]$ ;
16                 }
17             }
18      else
19      { // If  $P$  is not small, divide  $P$  into subproblems.
20        // Find where to split the set.
21         $mid := \lfloor (i + j)/2 \rfloor$ ;
22        // Solve the subproblems.
23        MaxMin( $i, mid, max, min$ );
24        MaxMin( $mid + 1, j, max1, min1$ );
25        // Combine the solutions.
26        if ( $max < max1$ ) then  $max := max1$ ;
27        if ( $min > min1$ ) then  $min := min1$ ;
28      }
29 }

```

Example: find max and min in the array:

22, 13, -5, -8, 15, 60, 17, 31, 47 (n = 9)

Index:	1	2	3	4	5	6	7	8	9
Array:	22	13	-5	-8	15	60	17	31	47



Analysis: For algorithm containing recursive calls, we can use recurrence relation to find its complexity

T(n) - # of comparisons needed for Rmaxmin

Recurrence relation:

$$\begin{cases} T(n) = 0 & n = 1 \\ T(n) = 1 & n = 2 \\ T(n) = 2T\left(\frac{n}{2}\right) + 2 & n > 2 \end{cases}$$

Assume $n = 2^k$ for some integer k

$$= 2^{k-1} T\left(\frac{n}{2^{k-1}}\right) + (2^{k-1} + 2^{k-2} + \dots + 2^1)$$

$$= 2^{k-1} \cdot T(2) + (2^k - 2) = \frac{n}{2} \cdot 1 + n - 2$$

$$= 1.5n - 2$$

When n is a power of two, $n = 2^k$ for some positive integer k , then

$$\begin{aligned} T(n) &= 2T(n/2) + 2 \\ &= 2(2T(n/4) + 2) + 2 \\ &= 4T(n/4) + 4 + 2 \\ &\vdots \\ &= 2^{k-1}T(2) + \sum_{1 \leq i \leq k-1} 2^i \\ &= 2^{k-1} + 2^k - 2 = 3n/2 - 2 \end{aligned}$$